

Documentación del Intérprete — Lenguaje FAF.net

Descripción General del Lenguaje

El lenguaje de programación FAF.net está compuesto por una sección de declaración de variables seguida de un cuerpo de programa. Las variables pueden ser de tipo **real**, **nat** (natural) o **matrix** (matriz de reales con dimensiones fijas). Todas las variables deben ser declaradas antes de ser utilizadas.

El cuerpo del programa se delimita con llaves (`{ }`) y contiene una secuencia de sentencias separadas por punto y coma. Las sentencias disponibles son: asignaciones, lecturas, escrituras, condicionales (`if/else`) y ciclos (`while`).

El lenguaje soporta expresiones aritméticas sobre reales (suma, resta, multiplicación, división, potencia y raíz), y expresiones sobre matrices (suma, resta, multiplicación entre matrices, transposición, producto escalar). Las condiciones se expresan mediante operadores relacionales (`<`, `>`, `==`, `>=`, `<=`, `<>`). El lenguaje es **sensible a mayúsculas y minúsculas**.

Descripción de la Semántica Asociada

Program: Representa la estructura completa del programa. Comienza con la palabra reservada `program` seguida del nombre del programa y la sección `var` con las declaraciones. Luego aparece la palabra `cuerpo` y el bloque de sentencias encerrado entre llaves.

Var: Declara las variables del programa. Cada declaración asigna un identificador a un tipo mediante el símbolo `=`. Las declaraciones terminan con punto y coma y se pueden encadenar.

Type: Define el tipo de una variable. Puede ser `real` (número real), `nat` (número natural, tratado internamente como real), o `matrix[N, M]` donde `N` y `M` son expresiones que determinan la cantidad de filas y columnas respectivamente (máximo 50x50).

ConjSentencias: Representa una secuencia de una o más sentencias, cada una separada por punto y coma. Permite que el cuerpo del programa ejecute múltiples acciones en secuencia.

B: Variable auxiliar de la gramática que permite encadenar conjuntos de sentencias. Deriva en otro ConjSentencias o en ϵ .

Sentencia: Representa una acción del programa. Puede ser una asignación, una lectura, una escritura, un condicional o un ciclo.

Asignacion: Asigna el resultado de una expresión a una variable. Si la variable es de tipo matriz, se puede asignar a una celda específica usando índices entre corchetes (`id[filas, cols] := expresion`). Si es de tipo real o nat, la asignación es directa (`id := expresion`).

C: Variable auxiliar de la gramática para la asignación. Distingue entre asignación directa (`:=`) y asignación indexada a una celda de matriz (`[expr, expr] := expr`).

Read: Sentencia de lectura. Muestra una cadena por pantalla como prompt y lee un valor numérico desde la entrada estándar. Puede leer un escalar (`read("mensaje", id)`) o una celda de matriz (`read("mensaje", id[filas, cols])`).

D: Variable auxiliar de la gramática para la lectura. Distingue entre lectura de variable escalar (cierra con `)`) y lectura de celda de matriz (índices entre corchetes antes del cierre).

Write: Sentencia de escritura. Imprime por pantalla una lista de uno o más elementos separados por comas: cadenas, expresiones reales o matrices completas.

ConjWrite: Representa un elemento o lista de elementos a imprimir. Puede comenzar con una cadena de texto o con una expresión real/matricial.

E: Variables auxiliares de la gramática que permiten encadenar múltiples elementos en la lista de escritura mediante comas.

Condicional: Estructura `if <cond> then { <sentencias> } con rama opcional else then { <sentencias> }`. Evalúa la condición y ejecuta el bloque correspondiente.

Y: Variable auxiliar que representa la rama opcional else del condicional. Si no hay rama else, deriva en ϵ .

While: Ciclo condicional `while <cond> { <sentencias> }`. Evalúa la condición y repite el bloque de sentencias mientras ésta sea verdadera.

Cond: Condición relacional formada por dos expresiones reales separadas por un operador relacional. Operadores disponibles: `<`, `>`, `==`, `>=`, `<=`, `<>`.

ExpresionReal: Expresión aritmética que evalúa a un valor real o matricial. Está formada por un término seguido opcionalmente de sumas o restas.

SumRes : Variable auxiliar que maneja las sumas (+) y restas (-) de la expresión. Permite asociatividad por izquierda.

Termino: Componente de una expresión real formado por un término secundario seguido opcionalmente de multiplicaciones o divisiones.

MultDiv : Variable auxiliar que maneja multiplicaciones escalares (*) y divisiones (/). Tiene mayor precedencia que las sumas y restas.

TerminoSecundario: Componente de mayor precedencia en la expresión. Puede ser $\text{sqrt}(\text{expr}, n)$ (raíz n -ésima), $\text{pot}(\text{base}, \text{exp})$ (potencia), una expresión entre paréntesis, o un operando simple.

Operando: Unidad básica de una expresión. Puede ser un identificador (con índices opcionales para matrices), un literal real, un literal natural, un operando negado (-), o una función matricial: $\text{add}(m1, m2)$, $\text{subtract}(m1, m2)$, $\text{multiplication}(m1, m2)$, $\text{transpose}(m)$, $\text{multesc}(m, \text{escalar})$.

I: Variable auxiliar para el acceso indexado a matrices. Si está presente, incluye dos índices [fila, col]. Si es épsilon, se accede a la variable completa.

CM: Constante matriz literal definida entre llaves externas que envuelven a M.

M: Representa una fila de la constante matriz, encerrada entre llaves internas. Seguida de J para permitir múltiples filas.

J: Variable auxiliar que permite encadenar múltiples filas en la constante matriz mediante comas.

Fila: Secuencia de expresiones reales separadas por comas que representan los valores de una fila de la matriz.

K: Variable auxiliar que permite encadenar múltiples columnas dentro de una fila mediante comas.

CantFilasDe: Función especial $\text{cantFilasDe}(\text{id})$ que devuelve el número de filas de la matriz identificada por id como valor real.

CantColumnasDe: Función especial $\text{cantColumnasDe}(\text{id})$ que devuelve el número de columnas de la matriz identificada por id como valor real.

Componentes del Intérprete

Analizador Léxico

El analizador léxico es responsable de transformar la secuencia de caracteres del archivo fuente en una secuencia de componentes léxicos (tokens). Implementa autómatas finitos determinísticos para reconocer los distintos tipos de tokens.

Tabla de Símbolos: Se inicializa con todas las palabras reservadas del lenguaje: program, var, cuerpo, real, nat, matrix, read, write, if, then, else, while, sqrt, pot, add, subtract, multiplication, transpose, multesc, cantfilde, cantcolde. Si un identificador leído no se encuentra en la tabla, se agrega con el tipo Tid.

Reconocimiento de identificadores: Se implementa mediante un AFD de 4 estados. Un identificador válido comienza con una letra (a-z, A-Z) y puede contener letras, dígitos y guiones bajos (no puede terminar en guion bajo ni contener dos guiones seguidos). La comparación con la tabla de palabras reservadas se realiza en minúsculas para garantizar insensibilidad de palabras clave.

Reconocimiento de números: Se implementa mediante un AFD de 4 estados. Distingue entre naturales (solo dígitos, estado final 1 → TNat) y reales (dígitos, punto decimal, dígitos, estado final 3 → Treal).

Reconocimiento de cadenas: Se implementa mediante un AFD de 3 estados. Una cadena válida está delimitada por comillas dobles ("). El lexema almacenado no incluye las comillas.

Reconocimiento de símbolos especiales: Se reconocen los símbolos {, }, [,], (,), ,, +, -, *, /, =, :, . Para los símbolos potencialmente dobles (:=, ==, >=, <=, <>), se lee un carácter adicional para determinar si forman un operador compuesto.

Analizador Sintáctico

El analizador sintáctico implementa un parser descendente predictivo no recursivo (LL(1)) basado en una **Tabla de Análisis Sintáctico (TAS)** y una pila explícita. Construye el árbol de derivación durante el análisis.

Tabla de Análisis Sintáctico (TAS): Es una tabla bidimensional indexada por variable gramatical y componente léxico (terminal). Cada celda contiene un puntero a la producción que debe aplicarse. Las celdas con cant = 0 representan producciones épsilon.

Árbol de Derivación: Se construye mediante nodos (tNodoArbol) que almacenan el símbolo gramatical, el lexema (para los terminales) y un arreglo de punteros a

los nodos hijos. El árbol se guarda en un archivo de texto al finalizar el análisis exitoso.

Algoritmo: El parser mantiene una pila de pares (símbolo, nodo del árbol). Inicialmente apila el símbolo de fin (TEnd) y la variable inicial (VProgram). En cada iteración desapila el tope: si es un terminal, lo compara con el token actual; si es una variable, consulta la TAS y apila los símbolos de la producción correspondiente, agregando los nodos hijos al árbol. El análisis termina con éxito cuando se desapila TEnd y el token actual también es TEnd.

Manejo de errores: Si el tope de la pila es un terminal que no coincide con el token actual, o si la TAS no tiene entrada para una variable con el token actual, se reporta error sintáctico indicando qué se esperaba y qué se encontró. Si el token actual es TError, se reporta error léxico.

Analizador Semántico / Evaluador

El evaluador recorre el árbol de derivación construido por el analizador sintáctico y ejecuta el programa. Implementa un **estado** que mantiene los valores de todas las variables declaradas.

Estado (TEstado): Arreglo de hasta 50 elementos, cada uno con: nombre del identificador, tipo (TrealL o TmatrizL), valor real y valor matricial. Las matrices se representan como arreglos bidimensionales de reales con sus dimensiones almacenadas.

Operaciones sobre matrices implementadas:

- SumaMatrices: suma elemento a elemento; requiere dimensiones iguales.
- RestaMatrices: implementada como suma con la segunda matriz multiplicada por -1.
- MultiplicacionMatrices: producto matricial estándar; requiere que columnas de la primera iguale filas de la segunda.
- TransposicionMatrices: transpone intercambiando filas y columnas.
- MultEscalar: multiplica todos los elementos de la matriz por un escalar.

Evaluación de expresiones: Las expresiones se evalúan recursivamente siguiendo la estructura del árbol. Cada función eval recibe el nodo del árbol y el estado, y devuelve el resultado como un par (valor real, valor matricial) junto con el tipo resultado. Las prioridades de operadores se respetan por la estructura jerárquica de la gramática: pot/sqrt > *// > +/-.

Escritura de matrices: Se imprime en formato $\{\{v_1, v_2, \dots\}, \{v_3, v_4, \dots\}, \dots\}$ con dos decimales por valor.

Gramática del Lenguaje

Gramática BNF

La forma BNF (Backus-Naur Form) es la representación canónica de la gramática, sin factorizar ni eliminar recursividad a izquierda. Permite visualizar la estructura completa del lenguaje.

```
<Program>      ::= "Program" "id" "var" <Var> "Cuerpo" "{" <ConjSentencias> "}"
<Var>          ::= "id" "=" <Type> ";" <Var>
                | ε
<Type>         ::= "real"
                | "nat"
                | "Matrix" "[" "Nat" ", " "Nat" "]"
<ConjSentencias> ::= <Sentencia> ";"
                | <Sentencia> ";" <ConjSentencias>
<Sentencia>     ::= <Asignacion>
                | <Read>
                | <Write>
                | <Condicional>
                | <While>
<Asignacion>    ::= "id" ":" <ExpresionReal>
                | "id" "[" <ExpresionReal> ", " <ExpresionReal> "]" ":" <ExpresionReal>
<Read>         ::= "read" "(" "string" ", " "id" ")"
                | "read" "(" "string" ", " "id" "[" <ExpresionReal> ", " <ExpresionReal> "]" ")"
<Write>        ::= "write" "(" <ConjWrite> ")"
<ConjWrite>     ::= "string"
                | "string" ", " <ConjWrite>
                | <ExpresionReal>
                | <ExpresionReal> ", " <ConjWrite>
<Condicional>   ::= "if" <Cond> "then" "{" <ConjSentencias> "}"
                | "if" <Cond> "then" "{" <ConjSentencias> "}" "else" "then" "{"
<ConjSentencias> ::= "}"
<While>        ::= "While" <Cond> "{" <ConjSentencias> "}"
<Cond>         ::= <ExpresionReal> "OpRel" <ExpresionReal>
<ExpresionReal> ::= <ExpresionReal> "+" <ExpresionReal>
                | <ExpresionReal> "-" <ExpresionReal>
                | <ExpresionReal> "*" <ExpresionReal>
```

```

| <ExpresionReal> "/" <ExpresionReal>
| "-" <ExpresionReal>
| "(" <ExpresionReal> ")"
| "sqrt" "(" <ExpresionReal> "," <ExpresionReal> ")"
| "pot" "(" <ExpresionReal> "," <ExpresionReal> ")"
| "add" "(" <ExpresionReal> "," <ExpresionReal> ")"
| "subtract" "(" <ExpresionReal> "," <ExpresionReal> ")"
| "multiplication" "(" <ExpresionReal> "," <ExpresionReal> ")"
| "transpose" "(" <ExpresionReal> ")"
| "multEsc" "(" <ExpresionReal> "," <ExpresionReal> ")"
| "cantFilasDe" "(" "id" ")"
| "cantColumnasDe" "(" "id" ")"
| "id"
| "id" "[" <ExpresionReal> "," <ExpresionReal> "]"
| "Real"
| "Nat"
| <CM>
<CM>      ::= "{" <M> "}"
<M>       ::= "{" <Fila> "}"
           | "{" <Fila> "}" "," <M>
<Fila>    ::= <ExpresionReal>
           | <ExpresionReal> "," <Fila>

```

Gramática LL(1)

La gramática LL(1) es la versión factorizada y sin recursividad a izquierda, apta para ser analizada por un parser descendente predictivo con un token de lookahead. Se introducen variables auxiliares (B, C, D, E, I, J, K, SumRes, MultDiv, etc.) para eliminar las ambigüedades y las recursiones izquierdas de la BNF.

```

<Program>  ::= "Program" "id" "var" <Var> "Cuerpo" "{" <ConjSentencias> "}"
<Var>      ::= "id" "=" <Type> "," <Var> | ε
<ConjSentencias> ::= <Sentencia> ";" <B>
<B>        ::= <ConjSentencias> | ε
<Type>     ::= "real" | "Matrix" "[" "Nat" "," "Nat" "]" | "nat"
<Sentencia> ::= <Asignacion> | <Read> | <Write> | <Condicional> | <While>
<Asignacion> ::= "id" <C>
<C>        ::= <ExpresionReal> | "[" <ExpresionReal> "," <ExpresionReal> "]" ":"
<ExpresionReal>
<Read>     ::= "read" "(" "string" "," "id" <D>
<D>        ::= ")" | "[" <ExpresionReal> "," <ExpresionReal> "]" ")"
<Write>    ::= "write" "(" <ConjWrite> ")"
<ConjWrite> ::= "string" <E> | <ExpresionReal> <E>

```

```

<E>          ::= "," <ConjWrite> | ε
<Condicional> ::= "if" <Cond> "then" "{" <ConjSentencias> "}" <Y>
<Y>          ::= "else" "then" "{" <ConjSentencias> "}" | ε
<While>      ::= "While" <Cond> "{" <ConjSentencias> "}"
<Cond>       ::= <ExpresionReal> "OpRel" <ExpresionReal>
<ExpresionReal> ::= <Termino> <SumRes>
<SumRes>     ::= "+" <ExpresionReal> | "-" <ExpresionReal> | ε
<Termino>    ::= <TerminoSecundario> <MultDiv>
<MultDiv>    ::= "*" <TerminoSecundario> <MultDiv> | "/" <TerminoSecundario>
<MultDiv> | ε
<TerminoSecundario> ::= "sqrt" "(" <ExpresionReal> "," <ExpresionReal> ")"
                    | "pot" "(" <ExpresionReal> "," <ExpresionReal> ")"
                    | "(" <ExpresionReal> ")"
                    | <Operando>
<Operando>    ::= "id" <I>
                    | "Real" | "Nat"
                    | "-" <Operando>
                    | "add" "(" <ExpresionReal> "," <ExpresionReal> ")"
                    | "subtract" "(" <ExpresionReal> "," <ExpresionReal> ")"
                    | "multiplication" "(" <ExpresionReal> "," <ExpresionReal> ")"
                    | "transpose" "(" <ExpresionReal> ")"
                    | "multEsc" "(" <ExpresionReal> "," <ExpresionReal> ")"
                    | <CM> | <CantFilasDe> | <CantColDe>
<CantFilasDe>  ::= "cantFilasDe" "(" "id" ")"
<CantColDe>   ::= "cantColumnasDe" "(" "id" ")"
<I>           ::= "[" <ExpresionReal> "," <ExpresionReal> "]" | ε
<CM>          ::= "{" <M> "}"
<M>           ::= "{" <Fila> "}" <J>
<J>           ::= "," <M> | ε
<Fila>        ::= <ExpresionReal> <K>
<K>           ::= "," <Fila> | ε

```

Programas de Ejemplo

Programa 7 — Multiplicación de dos matrices NxM

Lee dos matrices A (NxM) y B (MxP) elemento por elemento, las multiplica usando la función `multiplication` y muestra la matriz resultado.

Programa 8 — Normalización Min-Max por columna

Lee una matriz, calcula el mínimo y máximo de cada columna y normaliza cada valor según la fórmula: $\text{valor_norm} = (\text{valor} - \text{min}) / (\text{max} - \text{min})$. Muestra la matriz normalizada.

Programa 9 — Simetrización de una matriz

Lee una matriz cuadrada, luego calcula su **matriz simétrica** aplicando la fórmula: $S = (A + A^T) / 2$. Para ello transpone la matriz ingresada usando `transpose`, suma la original con su transpuesta usando `add`, y multiplica el resultado por el escalar 0.5 con `multesc`. Finalmente muestra la matriz original y la matriz simetrizada resultante.